SWINBURNE VIETNAM

HO CHI MINH CAMPUS

COS30018 - COS30018-Intelligent Systems

Oral Defense week 9

INSTRUCTOR: MR. KY TRUNG PHAM

STUDENT NAME: LÊ TIẾN DŨNG-SWS00683

HO CHI MINH CITY –2026

Contents

Oral Defense week 10 3

1. (Extension - Task C.7) Describe your Sentiment Analysis implementation. How do you convert textual news/social media data into a numerical "Sentiment Score"? 3

2. Explain the "Classification" model you built for the extension. Why is it sometimes easier to predict "Up/Down" (Binary) than the exact "Price" (Regression)? 3

3.(Lecture 9 Concept): Algorithmic trading is a classic use case for Reinforcement Learning (RL). If you were to transition your project from simply predicting prices to trading autonomously using RL, how would you define the "State", the "Action Space", and the "Reward"? 4

Oral Defense week 9

1. What is the "Ensemble" approach? How does combining a statistical model like ARIMA with a Deep Learning model like LSTM improve the reliability of the prediction?

The ensemble approach means I do not rely on only one model. Instead, I combine predictions from two different model families:

- a statistical time-series model like SARIMA/ARIMA
- a deep learning recurrent model like LSTM, GRU, or RNN

In my C.6 implementation, I combine them using a weighted average:

$$\hat{y}_{ens} = w \cdot \hat{y}_{DL} + (1 - w) \cdot \hat{y}_{SARIMA}$$

This is implemented in my ensemble_weighted(...) function.

The reason this can improve reliability is that the two models learn different kinds of patterns:

- SARIMA/ARIMA is strong at structured linear time-series behavior such as trend and seasonality
- LSTM/GRU/RNN can learn more complex nonlinear patterns from data

So, if one model is weak in some situations, the other can partly compensate. In my report, I found that ensemble performance depends on the relative strength of the two components and the ensemble weight. In my experiments, GRU was the strongest deep learning model, SARIMA alone had much higher error, and the best ensemble used a higher deep-learning weight.

In shorter word: The ensemble approach improves robustness by combining a traditional model and a deep learning model, so the final prediction does not depend on only one method. In my code, I use a weighted average, and the weight controls how much influence each model has.

2. Explain how you integrated ARIMA/SARIMA into your Python workflow. How do these traditional models handle "Seasonality" in stock data?

In my Python workflow, SARIMA is integrated in three clear steps:

1. Fit SARIMA on the training Close-price series only using fit_sarima(train_series, order, seasonal_order)

2. Forecast the full test horizon using forecast_sarima(fitted_model, steps)
3. Align SARIMA predictions with the deep learning predictions so both output the same number of test values before combining them in the ensemble

This is important because for ensembling, both models must predict the same timestamps and produce vectors of the same length. In my report, I explained that the DL model produces one prediction per test sample, and SARIMA forecasts exactly len(test) steps, so both can be evaluated against the same ground-truth vector.

For seasonality, I used SARIMA, not plain ARIMA, because SARIMA includes a seasonal_order = (P, D, Q, m) term in addition to the normal order = (p, d, q). In my report, I explained that:

- (p,d,q) controls autoregressive, differencing, and moving-average terms
- (P,D,Q,m) controls the seasonal part
- I used m = 5 to represent a weekly trading cycle of 5 trading days

So if the stock tends to show repeating weekly effects across trading days, SARIMA can model that through the seasonal component. In your experiments, the seasonal order was fixed to (1,0,1,5).

In shorter word:

I fit SARIMA on the training Close-price series, forecast the test period, and then align those predictions with the deep learning predictions before combining them. Seasonality is handled through the SARIMA seasonal parameters (P,D,Q,m), and I used m=5 to represent a 5-day trading week.

3. How do you resolve conflicts if the LSTM predicts a "Price Up" movement while the ARIMA model predicts a "Price Down" movement?

In my C.6 system, I do not resolve that conflict by majority vote. I resolve it using the weighted ensemble output.

That means even if the deep learning model implies price up and SARIMA implies price down, I still combine the two numeric price predictions first:

$$\hat{y}_{ens} = w \cdot \hat{y}_{DL} + (1 - w) \cdot \hat{y}_{SARIMA}$$

Then the final direction comes from the ensemble price, not from the individual model votes.

So the conflict is resolved by the weight:

- if w_dl is high, the deep learning model has more influence
- if w_dl is low, SARIMA has more influence

In my experiments, I tested weights such as 0.2, 0.5, and 0.8, and found that a higher DL weight worked better because SARIMA was weaker than the deep learning model in my dataset. The best ensemble used GRU + SARIMA(1,1,1)(1,0,1,5) with w_dl = 0.8 and achieved the lowest ensemble MAE.

So if LSTM says “up” and SARIMA says “down,” my answer is:

> I let the weighted ensemble decide. The final prediction is a weighted average of both predicted prices, and the model with the better validation performance should receive the larger weight. In my experiments, deep learning was stronger, so a higher w_dl resolved conflicts in favor of the DL model more often.